

Week 4: Customer Order Management and Order History System

- ATCHAYA [E25AI106]

Scenario:

The phone case business allows customers to place orders for different phone case products. The system stores overall order information, including the customer, order date, total amount, and order status. It also maintains individual product details for every order, such as product ID, quantity, and price. The system is used to manage customer orders and generate useful order history and purchase analysis reports.

1. Create Orders and Order_Details Tables

Orders:

```
SQL> CREATE TABLE Orders (Order_ID NUMBER PRIMARY KEY, Customer_ID VARCHAR2(5) NOT NULL, Order_Date DATE NOT NULL, Total_Amount NUMBER(10,2) NOT NULL, Order_Status VARCHAR2(20) NOT NULL, CONSTRAINT fk_orders_customer FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID), CONSTRAINT chk_order_status CHECK (Order_Status IN ('Pending','Shipped','Delivered')));
Table created.
```

Order_Details:

```
SQL> CREATE TABLE Order_Details (Order_Detail_ID NUMBER PRIMARY KEY, Order_ID NUMBER NOT NULL, Product_ID VARCHAR2(7) NOT NULL, Quantity NUMBER NOT NULL, Price NUMBER(10,2) NOT NULL, CONSTRAINT fk_od_order FOREIGN KEY(Order_ID) REFERENCES Orders(Order_ID), CONSTRAINT fk_od_product FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID), CONSTRAINT chk_od_quantity CHECK(Quantity > 0), CONSTRAINT chk_od_price CHECK(Price > 0));
Table created.
```

2. Inserting values:

Orders:

```
SQL> INSERT INTO Orders VALUES (1001, 'C001', DATE '2026-09-20', 1299.00, 'Pending');
1 row created.

SQL> INSERT INTO Orders VALUES (1002, 'C002', DATE '2026-09-21', 899.00, 'Shipped');
1 row created.

SQL> INSERT INTO Orders VALUES (1003, 'C003', DATE '2026-09-22', 1599.00, 'Delivered');
1 row created.

SQL> INSERT INTO Orders VALUES (1004, 'C001', DATE '2026-09-23', 699.00, 'Pending');|
```

Order_Details:

```
SQL> INSERT INTO Order_Details VALUES (1, 1001, 'P000001', 2, 499.00);
1 row created.

SQL> INSERT INTO Order_Details VALUES (2, 1001, 'P000002', 1, 301.00);
1 row created.

SQL> INSERT INTO Order_Details VALUES (3, 1002, 'P000003', 1, 899.00);
1 row created.

SQL> INSERT INTO Order_Details VALUES (4, 1003, 'P000004', 2, 799.50);
1 row created.

SQL> INSERT INTO Order_Details VALUES (5, 1004, 'P000005', 1, 699.00);
1 row created.
```

3. Viewing Table structure:

```
SQL> desc Orders;
Name                               Null?    Type
-----
ORDER_ID                           NOT NULL NUMBER
CUSTOMER_ID                        NOT NULL VARCHAR2(5)
ORDER_DATE                         NOT NULL DATE
TOTAL_AMOUNT                       NOT NULL NUMBER(10,2)
ORDER_STATUS                       NOT NULL VARCHAR2(20)

SQL> desc Order_Details;
Name                               Null?    Type
-----
ORDER_DETAIL_ID                    NOT NULL NUMBER
ORDER_ID                           NOT NULL NUMBER
PRODUCT_ID                         NOT NULL VARCHAR2(7)
QUANTITY                          NOT NULL NUMBER
PRICE                             NOT NULL NUMBER(10,2)
```

4. Viewing Values in Table:

Orders:

```
SQL> select * from Orders;

ORDER_ID CUSTO ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-----
1001 C001 20-SEP-26 1299 Pending
1002 C002 21-SEP-26 899 Shipped
1003 C003 22-SEP-26 1599 Delivered
1004 C001 23-SEP-26 699 Pending
1005 C004 24-SEP-26 450 Shipped
1006 C005 24-SEP-26 750 Pending
1007 C002 25-SEP-26 1200 Delivered
1008 C003 25-SEP-26 550 Shipped
1009 C005 26-SEP-26 999 Pending
1010 C004 24-SEP-26 550 Shipped
1011 C004 24-SEP-26 450 Shipped

11 rows selected.
```

Order_Details:

```
SQL> select * from Order_Details;
```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT	QUANTITY	PRICE
1	1001	P000001	2	499
2	1001	P000002	1	301
3	1002	P000003	1	899
4	1003	P000004	2	799.5
5	1004	P000005	1	699
6	1005	P000001	1	450
7	1006	P000002	2	375
8	1007	P000003	1	1200
9	1008	P000004	1	550
10	1009	P000005	1	999
11	1010	P000005	1	990

```
11 rows selected.
```

5. Establish Relationships

Customer and Orders

- One customer can place multiple orders.
- Each order belongs to one customer.

Relationship:

Customer (1) → Orders (Many)

The Customer_ID in the Orders table is used as a foreign key referencing the Customer table.

Orders and Products

- One order can contain multiple products.
- One product can appear in multiple orders.

This creates a many-to-many relationship between Orders and Products.

Relationship:

Orders (Many) ↔ Products (Many)

This relationship is implemented using the Order_Details table.

6. Apply Constraints:

Applied Primary key, Foreign key, NOT NULL, check while creating table itself.

Primary Key

- Customer_ID is the primary key in the Customer table.
- Order_ID is the primary key in the Orders table.
- Order_Detail_ID is the primary key in the Order_Details table.
- Product_ID is the primary key in the Product table.

Foreign Key

- Orders.Customer_ID references Customer.Customer_ID.
- Order_Details.Order_ID references Orders.Order_ID.
- Order_Details.Product_ID references Product.Product_ID.

NOT NULL

Important attributes such as customer ID, order date, total amount, order status, quantity, and price are defined as NOT NULL so that essential information cannot be left empty.

CHECK Constraint

The order status is restricted to valid values:

- Pending
- Shipped
- Delivered

The quantity and price are also checked to ensure that they are greater than zero.

7. Update and Delete Records:

```

SQL> UPDATE Orders SET Total_Amount = 500, Order_Status = 'Delivered' WHERE Order_ID = 1011;
1 row updated.

SQL> DELETE FROM Orders WHERE Order_ID = 1010;
DELETE FROM Orders WHERE Order_ID = 1010
*
ERROR at line 1:
ORA-02292: integrity constraint (C##PHONECASE.FK_OD_ORDER) violated - child
record found

SQL> DELETE FROM Order_Details WHERE Order_ID = 1010;
1 row deleted.

SQL> DELETE FROM Orders WHERE Order_ID = 1010;
1 row deleted.

SQL> UPDATE Order_Details SET Quantity = 2, Price = 1000 WHERE Order_Detail_ID = 9;
1 row updated.

```

8. Report 1: Customer Order History

Displays:

- Customer Name
- Order ID
- Order Date
- Total Amount
- Order Status

QUERY: SELECT Customer.Customer_Name, Orders.Order_ID, Orders.Order_Date, Orders.Total_Amount, Orders.Order_Status

FROM Customer

JOIN Orders

ON Customer.Customer_ID = Orders.Customer_ID;

-This report combines customer and order information to display the complete order history of customers.

```

SQL> SELECT Customer.Customer_Name, Orders.Order_ID, Orders.Order_Date, Orders.Total_Amount FROM Customer JOIN Orders ON Customer.Customer_ID = Orders.Customer_ID;

```

CUSTOMER_NAME	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT
Arun Kumar	1004	23-SEP-26	699
Arun Kumar	1001	20-SEP-26	1299
Aakash	1002	21-SEP-26	899
Rahul Raj	1003	22-SEP-26	1599

9. Report 2: Product-wise Order Report

Finds:

Number of times each product was ordered.

Total quantity of each product sold.

```
QUERY: SELECT Product.Product_ID, Product.Product_Name,  
COUNT(Order_Details.Order_ID) AS Times_Ordered,  
SUM(Order_Details.Quantity) AS Total_Quantity_Sold  
FROM Product  
JOIN Order_Details  
ON Product.Product_ID = Order_Details.Product_ID  
GROUP BY Product.Product_ID, Product.Product_Name;
```

-This report helps identify how frequently each product is ordered and the total number of units sold.

10. Report 3: Customer Purchase Analysis

❖ Customers with Maximum Orders:

```
QUERY: SELECT Customer.Customer_ID, Customer.Customer_Name,  
COUNT(Orders.Order_ID) AS Total_Orders  
FROM Customer  
JOIN Orders  
ON Customer.Customer_ID = Orders.Customer_ID  
GROUP BY Customer.Customer_ID, Customer.Customer_Name  
ORDER BY Total_Orders DESC;
```

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, COUNT(Orders.Order_ID) AS Total_Orders FROM Customer JOIN Orders ON Customer.Customer_ID = Orders.  
Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name ORDER BY Total_Orders DESC;
```

CUSTO	CUSTOMER_NAME	TOTAL_ORDERS
C005	Warthik S	2
C003	Rahul Raj	2
C001	Arun Kumar	2
C002	Aakash	2
C004	Divya Kumar	2

This report calculates the number of orders placed by each customer.

❖ Total Spending by Each Customer:

```
QUERY: SELECT Customer.Customer_ID, Customer.Customer_Name,
```

SUM(Orders.Total_Amount) AS Total_Spending

FROM Customer

JOIN Orders

ON Customer.Customer_ID = Orders.Customer_ID

GROUP BY Customer.Customer_ID, Customer.Customer_Name;

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, SUM(Orders.Total_Amount) AS Total_Spending FROM Customer JOIN Orders ON Customer.Customer_ID = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name;
```

CUSTO	CUSTOMER_NAME	TOTAL_SPENDING
C005	Warthik S	1749
C003	Rahul Raj	2149
C004	Divya Kumar	950
C002	Aakash	2099
C001	Arun Kumar	1998

-This report calculates the total amount spent by each customer

❖ Average Order Value:

QUERY: SELECT Customer.Customer_ID, Customer.Customer_Name,

AVG(Orders.Total_Amount) AS Average_Order_Value

FROM Customer

JOIN Orders

ON Customer.Customer_ID = Orders.Customer_ID

GROUP BY Customer.Customer_ID, Customer.Customer_Name;

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, AVG(Orders.Total_Amount) AS Average_Order_Value FROM Customer JOIN Orders ON Customer.Customer_ID = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name;
```

CUSTO	CUSTOMER_NAME	AVERAGE_ORDER_VALUE
C005	Warthik S	874.5
C003	Rahul Raj	1074.5
C004	Divya Kumar	475
C002	Aakash	1049.5
C001	Arun Kumar	999

-This report calculates the average value of orders placed by each customer.

